life.augmented

AN5659

Application note

Safety self-test library Arm® Cortex®-M integration tips

Introduction

This document helps the end user to correctly integrate the self-test library in its environment, by giving some hints and reminders. It is useful for SW architects, integrators and developers.

The X-CUBE-STL self-test library (STL) is an application-independent software released by ST to implement a relevant subset of safety mechanisms required by the safety concepts applicable to STM32 microcontrollers and microprocessors.

STL is an autonomous software, which executes on application demand selected tests to detect HW issues and reports the results to the application. It is a compilation tool chain-agnostic, so it can be run on any standard C-compiler.

STL is delivered in object code for the library itself, and in source code for the user interfaces definition and user parameters settings.

The X-CUBE-STL is built on STM32Cube software technology.

1 General information

This document applies to Arm®-based device X-CUBE-STL.

Note: Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.

1.1 Related safety documents

More informations about safety of STM32 products are available in:

- MCU safety manuals
- MCU FMEA
- MCU FMEDA
- STL safety manuals
- STL user guides

2 Acronyms

- STL: Self test library
- API: Application programming interface
- TM: Test module
- SVC: Supervisor call
- OS: Operating system

3 STL scheduling

For further details on scheduler principle refer to the dedicated section of the user guides.

3.1 STL API

The STL API is called in polling mode.

After a call to a STL function, the end user must:

- wait for this STL function return before calling another STL function
- check the execution test results of this STL function, provided:
  - directly by the API return value
  - indirectly via value modified in status list structure allocated by the end user (refer to the user structures section in the user guides):
    - STL_TmStatus_t for single test
    - STL_TmListStatus_t for Multiple test.

The test modules dedicated for testing memories require specific APIs function calls for initialization and configuration of the testing sequences. The configuration and the running APIs are using data stored at specific memory configuration structures allocated by the end user.

3.2 STL test

There are two ways to launch the STL tests:

1. **Single test**

   Each TM is managed (initialization, configuration, execution) individually via dedicated functions calls

2. **Multiple tests**

   The user enables the TMs (from the full TM list) to execute and then calls only one function to initialize, configure or execute the enabled TMs.

3.2.1 Mixing single and multiple tests

Single tests and multiple tests can be mixed inside the application. For example, this makes possible:

- to use multiple tests when there is no execution timing constraint
- to use single tests when there are execution timing constraints.

This mix can also be used during multiple test execution to reset only the Flash memory test module or the RAM test module with single test interface.

The use of the corresponding APIs is well detailed in the dedicated sections test examples (multiple testing example), Flash memory multiple test example, RAM multiple test example and state machines of the user guides.

4 STL and interrupts - System responsive time

For further details, refer to the different sections about interrupt in the user guides.

The STL can be interrupted at any time. During some TMs, the STL masks the interrupts during the smallest data granularity time, fixed and mastered by the STL. Refer to STL user guide of the product to check the concerned TMs and their associated maximum interrupts masking time value.

Before running the STL test modules that masks the interrupts, the user must check that the STL interrupts masking durations are compatible with its own timing constraints. The user must schedule these STL test modules, only when STL interrupts masking time is not impacting the system performance (for more details on STL interrupt masking time refer to the dedicated section of the user guides).

5 STL privilege level and mode execution

5.1 Privilege level

For futher details on privileged-level, refer to the dedicated section of the user guide, where there are two options:

- If there is a detailed list of TMs to be executed with the privileged level, it means that other TMs can be executed with unprivileged level or privileged level.
- If there is no detailed list of TMs, it means that the whole STL must be executed with the privileged level.

5.2 Mode execution

For further details on processor mode refer to the dedicated section of the user guides (if available), where there are two options:

- If some TMs that require to be executed in Thread mode are mentioned, it means that other TMs can be executed in Thread mode or Handler mode.
- If there is no information for such TMs (section processor mode not available), the TMs can be executed in Thread mode or Handler mode

6 Memory

6.1 STL memory usage

6.1.1 Memory footprint

The STL memory footprint (RO code + RO data + RW data) is given for each product in the relevant section STL code and data size of the user guide. The user must ensure to have enough memory space.

6.1.2 Stack usage

The user guide, in corresponding section, gives information about the maximum STL stack usage. The user must ensure enough stack space.

6.2 RAM test information

6.2.1 Enable and disable dynamically the RAM backup buffer

This chapter supplements the information of RAM test and RAM backup buffer sections in the user guide.

When the flag STL_DISABLE_RAM_BCKUP_BUF is enabled there is no RAM backup buffer. This is chosen at compilation time. In addition, there is a way to use or not dynamically the RAM backup buffer, during execution time:

- Compile without enabling the flag `STL_DISABLE_RAM_BCKUP_BUF`, to enable the use the RAM backup buffer in the application
- Store the initial RAM backup buffer address (store `STL_pRamTmBckUpBuf` value)
- Set `STL_pRamTmBckUpBuf` to null, when the RAM backup buffer is not needed for some specific tests, (same as in `stl_user_param_template.c` file)

  ```
  STL_pRamTmBckUpBuf = NULL;
  ```

- Change RAM state value to `RAM_IDLE` or to `RAM_INIT` (use specific APIs; see also state machine section of the user guide) to restart completely the RAM test
- Use APIs to execute the RAM test without the backup buffer
- Restore the initial RAM backup buffer address
- Change RAM state value to `RAM_IDLE` or to `RAM_INIT` (use specific APIs; see also state machine section of user guide) to restart completely the RAM test
- Use APIs to execute the RAM test with the backup buffer

6.2.2 CPU TM12 (cache management) and RAM segmentation

This chapter supplements the information of the corresponding section in the user guide, if available.

6.2.2.1 Constraints

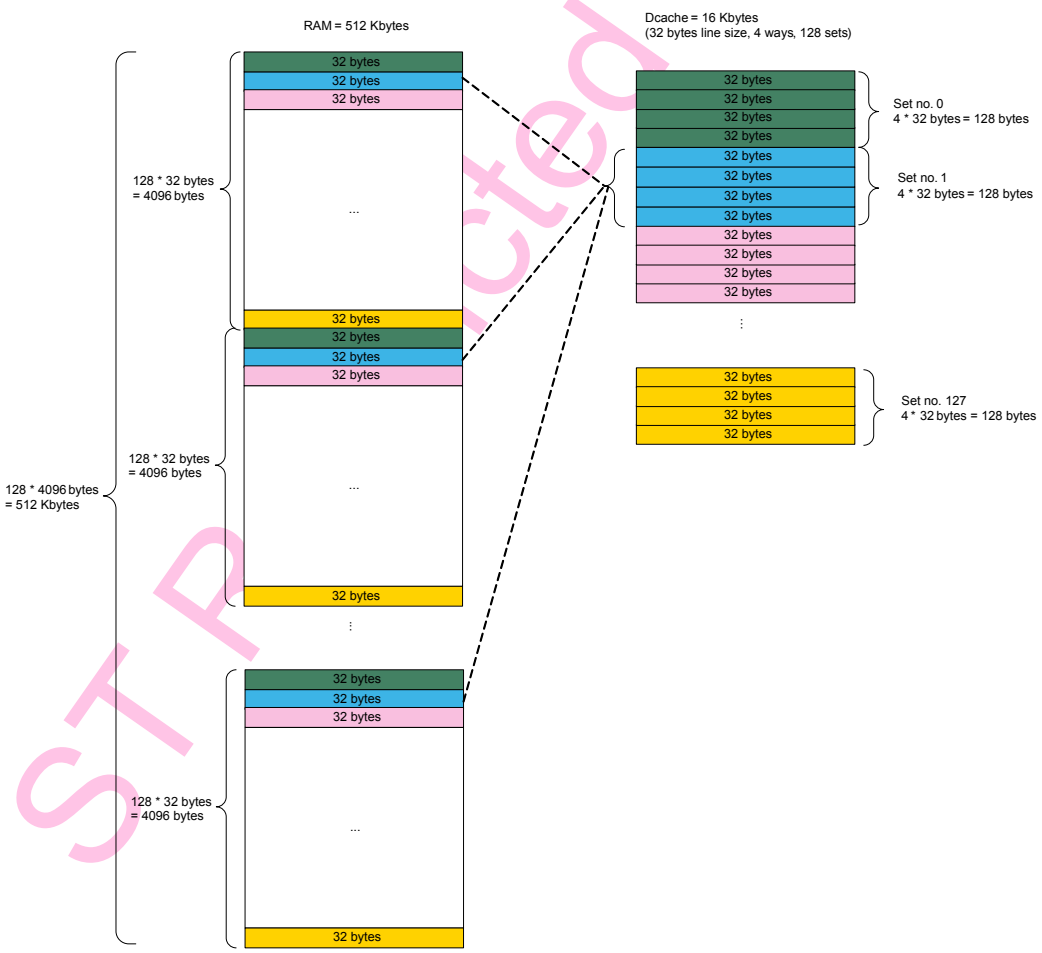
The design of STL CPU TM12 requires to control the content of a given data cache set. For this purpose, the STL implies the following constraint:

- All RAM areas associated to a given data cache set must be reserved for the STL. This implies that the user available RAM area is fragmented (see Data cache organization).

6.2.2.2 *Data cache organization*

Figure 1 shows how a RAM address is associated to a data cache set (STM32H7 memory configuration).

Figure 1. RAM address associated to a data cache set



6.2.2.3 *User consequence*

The maximum size of contiguous RAM area available for the user = data cache line size * (number of data cache sets – 1) = 4064 bytes in STM32H7 configuration.

6.2.2.4 *Linker scripts*

We choose the data cache set no.1 for the STL CPU TM12. The RAM areas associated to the data cache set no.1 are all the blue RAM areas.

The purpose is:

- to prevent the user to access all the blue RAM area
- to place the “stl_cpu_tm12_section” in one blue RAM area

EWARM linker script

It is simple to create an occurrence of a region, add or subtract a region.

Based on STM32H7:

```
define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
define symbol __ICFEDIT_region_RAM_end__ = 0x2007FFFF;
define symbol DCache_line_size = 32; /* fix value for Cortex M7 */
define symbol DCache_set_num = 128; /* fix value for STM32H7 */
define symbol Ram_DCache_set_occurence = (__ICFEDIT_region_RAM_end__ -
__ICFEDIT_region_RAM_start__ + 1) / (DCache_set_num * DCache_line_size);
define symbol Ram_DCache_set_displacement = DCache_set_num * DCache_line_size;
define symbol region_RAM_STL_CPU_TM12_start = __ICFEDIT_region_RAM_start__ +
DCache_line_size; /* DCache Set n°1 */

/* define region belonging to Set n°1 of DCache */
define region STL_CPU_TM12_region = mem:[from region_RAM_STL_CPU_TM12_start size
DCache_line_size repeat Ram_DCache_set_occurence displacement Ram_DCache_set_displacement];

/* define RAM regions available for user */
define region RAM_region = mem:[from __ICFEDIT_region_RAM_start__ to
__ICFEDIT_region_RAM_end__] - STL_CPU_TM12_region;
```

GCC linker script

It is not possible to create an occurrence of a region, add or subtract a region, based on STM32H7:

- For STL needs, we have to define one region to place the section `stl_cpu_tm12_section`

```
RAM_STL_CPU_TM12 (xrw) : ORIGIN = 0x20020020, LENGTH = 32
```

The user available RAM is defined with all the contiguous RAM areas between blue RAM areas. The read/write data are placed in these RAM regions:

```
RAM_0 (xrw)             : ORIGIN = 0x20000000, LENGTH = 32
RAM_1 (xrw)             : ORIGIN = 0x20000040, LENGTH = 4064
RAM_2 (xrw)             : ORIGIN = 0x20001040, LENGTH = 4064
RAM_3 (xrw)             : ORIGIN = 0x20002040, LENGTH = 4064
…..
RAM_127 (xrw)         : ORIGIN = 0x2007E040, LENGTH = 4064
RAM_128 (xrw)         : ORIGIN = 0x2007F040, LENGTH = 4032
```

6.2.2.5 *Workaround to avoid RAM segmentation*

To relax the constraint on RAM contiguous areas (i.e., in STM32H7, to have more than 4064 bytes of RAM contiguous area, the limit is the available RAM), follow these steps:

- Place `stl_cpu_tm12_section` in a cacheable RAM area;
- Execute the CPU TM12 only in single test. It means that CPU TM12 must be disabled when launching multiple tests, and so never executed in multiple tests;
- Disable IRQ when executing CPU TM12 single test. User must take care of its system responsiveness.

7 CRC information

This chapter supplements the information of sections Flash memory tests and CRC resources of the user guides.

The CRC calculation used by the scheduler or for Flash memory test is not based on default HW CRC configuration. The CRC used configuration for STL is defined in `stl_util.c`, inside `STL_UTIL_CRC_Init` function, and it has not be changed.

7.1 CRC and Flash memory test information

CRC calculation and flash memory test detailed description

This section describes the behavior of the ST CRC tool (CLI command line). If end user is using its own tool, it must behave the same:

- CLI checks if each binary within the elf file is 32-bit aligned. Otherwise, it performs a 32-bit alignment (with pattern 0x00). The end of each binary can be changed between the link output and the CLI output due to the 32-bit alignment.
- CLI computes CRCs for each binary of the elf file and only for the binary size. A CRC is calculated for every 1024 bytes (complete section, FLASH_SECTION_SIZE) and possibly for the last part of the binary less than 1024 bytes but aligned on 32 bits (incomplete section). Each time the computed CRC value is added to its corresponding address at the end of the Flash memory, within the CRC area. It means that the CLI output file, elf file, is modified by adding the CRC area.
- Customer application defines the Flash memory subsets to be tested. When testing an incomplete section, it is mandatory to adjust the final address of the subset to be exactly the end of the binary. It is done by checking directly the memory (or by checking the information from the elf file available after running the CLI). The information from the mapping file, the output from the link, may be incorrect for the final address in case of 32-bit alignment done by the CLI. It may be necessary to loop compiling and tuning to get the correct configuration of subsets against binaries. Of course, every time the application / binary is changed, the incomplete section needs to be fine-tuned.
- During execution time, STL computes the CRCs based on subsets configured in the application, for each section and potentially for the incomplete section. A CRC comparison is made between CLI pre-calculated value and STL calculated value.

Example of incomplete section and atomic behavior

STM32 IC with 512 KBytes Flash memory:

- Flash memory start @ 0x08000000
- Flash memory end @ 0x0807FFFF

After build/link and post-build script (CLI), output elf file with:

- Two binaries, 32-bit aligned by CLI
- CRC area, created by CLI

Table 1. Information extracted from elf file

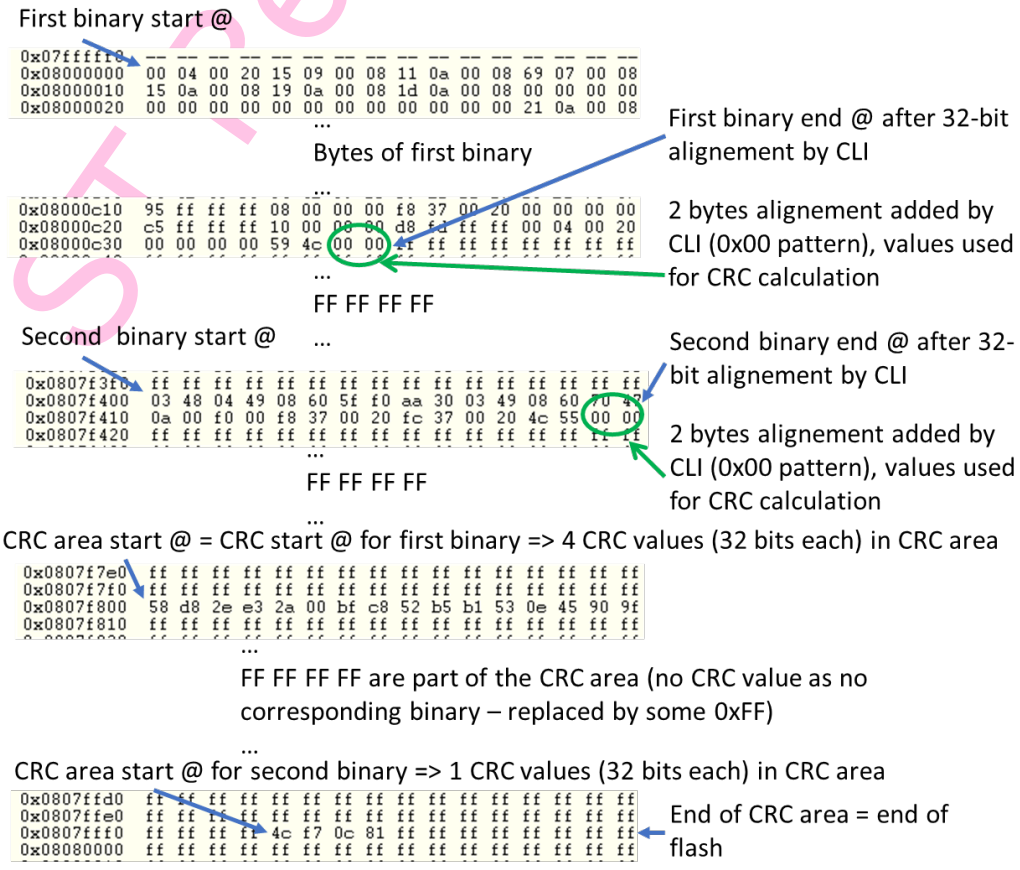
	Physical address	File size	Memory size	End address
1st binary	0x08000000	3218	3218	0x08000C37
2nd binary	0x0807F400	32	32	0x0807F41F
CRC area	0x0807F800	2048	2048	0x0807FFFF

Application Flash memory subsets configuration if the two full binaries must be tested:

- `FlashSubset2.StartAddr = 0x0807F400;`
- `FlashSubset2.EndAddr = 0x0807F41F;`
- `FlashSubset2.pNext = NULL;`
- `FlashSubset1.StartAddr = 0x08000000;`
- `FlashSubset1.EndAddr = 0x08000C37;`
- `FlashSubset1.pNext = &FlashSubset2;`
- `FlashConfig.pSubset = &FlashSubset1;`
- `FlashConfig.NumSectionsAtomic = AtomicAppliValue;`
- first subset:
  - 3 full sections (0x400 bytes size) + 1 incomplete section (0x38 bytes size)
  - 4 CRC values (32 bits each) in CRC area
- Second subset:
  - 1 incomplete section (0x20 bytes size)
  - 1 CRC values (32 bits each) in CRC area

If `AtomicAppliValue` ≥ 5, the two subsets are fully tested in one shot (from atomic view: 1 complete section = 1 incomplete section = 1 round).

Figure 2. Corresponding memory view



8 Dual core information

STL architecture is the same used for single core product, with one dedicated STL for each core.

For more details refer to the dual core user guide.

The additional constraints specific to the dual core are related to:

- shared resources between the two CPUs, RCC and CRC,
- RAM test

Note: Usually the safety concept for STM32 dual core microcontrollers imposes additional requirements to the individual execution of the STL (for instance, the exchange of STL results between the two CPUs). It is recommended to check related MCU safety manual to get guidance.

9 STL and Cortex-M33 with TrustZone

Refer to the user guide of Series with TrustZone for more information.

STL integration differs depending on the use of Cortex-M33:

- TrustZone is not activated. The STL and STL library are integrated in the same way as for others Cortex-M.
- TrustZone is activated. The STL library must be embedded once in secure binary. If the STL needs to be executed from non-secure state, then it must be called via the SG (secure gateway) feature.

The STL design is made for using the same STL library in case TrustZone is activated, but also in case TrustZone is not activated.

It is why, for security reason:

- the use of RAM backup buffer is mandatory. The flag STL_DISABLE_RAM_BCKUP_BUF, to disable the RAM backup buffer, is not available for Cortex-M33. An additional check of the RAM backup buffer address is added for Cortex-M33. It must be located in secure RAM when TrustZone is activated. It is not possible to set the RAM backup buffer address to NULL to execute without the RAM backup buffer, as it is described in Section 6.2.1 Enable and disable dynamically the RAM backup buffer.
- the addition of user tests feature in the STL scheduler is not available for Cortex-M33, to avoid potential execution of some non-secure code from secure state, that could expose to security breaches.

The applications in the release package are delivered with TrustZone activated.

10 Compiler information

10.1 STL compliance

For further details on compliances, refer to the dedicated section of the user guides.

For example, warning on `wchar_t` appears when the STL library is generated with IAR EWARM v7.x (`wchar_t` set to 16 bits) and compiled with the compiler/linker user application that set `wchar_t` to 32 bits. This warning has no impact on STL behavior.

10.2 Arm Compiler 6 support information

The support of Keil Arm Compiler 6 is not yet integrated in each X-CUBE-STL release. It can be added in STL sources files following these steps:

In `stl_param_template.c` file

- replace

  ```
  #elif defined (__CC_ARM ) || defined(__GNUC__)
  ```

  by

  ```
  #elif defined (__CC_ARM ) || defined(__GNUC__) || (defined (__ARMCC_VERSION) && (__ARMCC_VERSION >= 6010050))
  ```

- replace, only for targets that supports CPU TM12

  ```
  #elif defined(__GNUC__)
  ```

  by

  ```
  #elif defined(__GNUC__) || (defined (__ARMCC_VERSION) && (__ARMCC_VERSION >= 6010050))
  ```

In `stl_util.c` file

- replace 3 times

  ```
  #elif defined(__GNUC__)
  ```

  by

  ```
  #elif defined(__GNUC__) || (defined (__ARMCC_VERSION) && (__ARMCC_VERSION >= 6010050))
  ```

11 Tips on main integration support requests

ST standard support logs indicate that the major part of STL integration support requests concerns the RAM and Flash memory tests where the subsets are wrongly defined. It is mandatory to satisfy the subsets definition constraints. For more details on Flash memory tests and RAM tests refer to the relevant sections of the user guides. For more information on Flash memory subset, refer also to section Section 7.1 CRC and Flash memory test information.

12 STL integration in a RTOS

Here are some tips to integrate the STL in an RTOS.

12.1 STL execution in a RTOS thread

STL is executed in a dedicated RTOS thread:

- Thread priority must be set accordingly (usually: idle task with lowest priority).
- Thread privilege level must be set accordingly:
  - privileged level is set permanently for this thread

  or
  - unprivileged is set for this Thread. Privileged level is temporary set (e.g. via SVC) for the required TMs if mentioned in the STL user manual.

Caution: The maximum interrupt servicing delay allowed by the system must be higher than the STL maximum interrupts masking time.

12.2 STL execution in Handler mode

STL is called in an interrupt context. The chosen interrupt priority must guarantee the correct RTOS behavior.

13 MISRA-C 2012

The STL fulfills MISRA-C 2012 rules with two derogations for the STL files delivered in source:

- D.4.3 in `stl_util.c` with:
  - `asm("CPSID i");`
  - `asm("MRS %0, PRIMASK" : "=r"(result))`
  - `asm("MSR PRIMASK,R0");`

  Derogation details: the use of assembler instructions to enable/disable interrupts (instead of CMSIS wrapper) is mixed with code in order to be:
  - CMSIS agnostic
  - independent from compiler in case of the use of compiler intrinsic functions.
- R.8.4 in `stl_user_param_template.c` with

  Definition of externally linked `STL_RomStart`, `STL_RomEnd`, `STL_RamTmBckUpBuf` have no compatible declarations.

  Derogation details: all variables are declared in `stl_user_param.h` file which is not included in `stl_user_param_template.c` because stl_user_param.h is not delivered to user, despite used in STL Library building.

Revision history

Table 2. **Document revision history**

Date	Version	Changes
29-Jun-2021	1	Initial release.

Contents

List of tables

List of figures